

```
In [ ]: !apt-get install -y libgl1-mesa-glx libegl1-mesa libgles2-mesa-dev libosmesa6-dev
!pip install mujoco mediapy matplotlib numpy -q
```

```
In [ ]: import os
os.environ["MUJOCO_GL"] = "egl"

import numpy as np
import mujoco
import mediapy as media
import matplotlib.pyplot as plt

xml = """
<mujoco model="one_dof_fishing_rod_3d_fish_motion">
  <compiler angle="degree" coordinate="local"/>
  <option timestep="0.01" gravity="0 0 -9.81"/>

  <visual>
    <global offwidth="960" offheight="720"/>
    <headlight diffuse="0.6 0.6 0.6" ambient="0.3 0.3 0.3" specular="0 0 0"/>
    <rgba haze="0.15 0.25 0.35 1"/>
  </visual>

  <asset>
    <texture type="skybox" builtin="gradient"
      rgb1="0.3 0.5 0.7" rgb2="0 0 0"
      width="512" height="3072"/>
    <texture type="2d" name="groundplane" builtin="checker"
      mark="edge" rgb1="0.2 0.3 0.4" rgb2="0.1 0.2 0.3"
      markrgb="0.8 0.8 0.8" width="300" height="300"/>
    <material name="groundplane" texture="groundplane"
      texuniform="true" texrepeat="5 5" reflectance="0.2"/>

    <material name="blue" rgba="0.1 0.2 0.9 1"/>
    <material name="cyan" rgba="0.0 0.8 0.9 1"/>
    <material name="gray" rgba="0.5 0.5 0.5 1"/>
    <material name="water" rgba="0.2 0.5 0.9 0.10"/>
  </asset>

  <worldbody>
    <light pos="0 -3 4" dir="0 1 -1"/>
    <geom name="floor" type="plane" size="0 0 0.05" material="groundplane"/>

    <!-- Visual-only water surface -->
    <geom name="water_surface" type="box" pos="1.4 0 0.0"
      size="2.4 1.2 0.01" material="water"
      contype="0" conaffinity="0"/>

    <body name="base" pos="0 0 0.10">
      <geom type="box" size="0.08 0.08 0.08" material="gray"/>

      <!--
      1-DOF vertical rod:
      q = 0 deg: rod vertical upward
      q increases: rod lowers forward in +x direction
      -->
    </body>
  </worldbody>
</mujoco>
"""
```

```

-->
<body name="rod" pos="0 0 0">
  <joint name="pitch" type="hinge" axis="0 1 0"
    limited="true" range="5 75" damping="1.8" armature="0.03"/>
  <geom name="rod_geom" type="capsule"
    fromto="0 0 0 0 0 1.0"
    size="0.025" material="blue"/>
  <site name="rod_tip" pos="0 0 1.0" size="0.035" rgba="1 0 0 1"/>
</body>
</body>

<!-- Fish target controlled from Python -->
<body name="fish" mocap="true" pos="1.8 0.45 -0.50">
  <geom name="fish_geom" type="sphere" size="0.10"
    material="cyan" contype="0" conaffinity="0"/>
</body>
</worldbody>

<actuator>
  <motor name="pitch_torque" joint="pitch" ctrllimited="true" ctrlrange="-
</actuator>
</mujoco>
"""

model = mujoco.MjModel.from_xml_string(xml)
data = mujoco.MjData(model)

PITCH_JOINT_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_JOINT, "pitch")
PITCH_QPOS_ID = model.jnt_qposadr[PITCH_JOINT_ID]

ROD_TIP_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_SITE, "rod_tip")
FISH_BODY_ID = mujoco.mj_name2id(model, mujoco.mjtObj.mjOBJ_BODY, "fish")
FISH_MOCAP_ID = model.body_mocapid[FISH_BODY_ID]

renderer = mujoco.Renderer(model, height=720, width=960)

cam = mujoco.MjvCamera()
cam.type = mujoco.mjtCamera.mjCAMERA_FREE
cam.lookat[:] = [1.10, 0.05, 0.20]
cam.distance = 3.0
cam.azimuth = 135
cam.elevation = -12

print(f"MuJoCo {mujoco.__version__} | nq={model.nq} nv={model.nv} nu={model.nu}")
print("Setup complete.")

```

MuJoCo 3.9.0 | nq=1 nv=1 nu=1
Setup complete.

```

In [ ]: DT = model.opt.timestep
        T_FINAL = 18.0
        DT_FRAME = 0.04

        STATE_MONITOR = 0
        STATE_HOOK_SET = 1
        STATE_REGULATE = 2

```

```

T_DESIRED = 1.0
T_LOW = 0.82
T_HIGH = 1.18

BITE_TIME = 2.4
HOOK_DURATION = 1.2

# Large enough motion for presentation
Q_INIT = np.deg2rad(48.0)
Q_HOOK = np.deg2rad(14.0)
Q_NEUTRAL = np.deg2rad(40.0)

Q_MIN = np.deg2rad(6.0)
Q_MAX = np.deg2rad(72.0)

FISH_BASE = np.array([1.80, 0.45, -0.50])

FILTER_ALPHA = 0.055
PITCH_QVEL_ID = 0 # Update this index based on your MuJoCo model

# Stronger response, but with asymmetric controller below
TENSION_TO_ANGLE_GAIN = np.deg2rad(42.0)
QEQ_ALPHA = 0.055
MAX_QEQ_STEP = np.deg2rad(0.22)

# Rod effect on simulated load-cell tension
ROD_TENSION_GAIN = 3.05
TENSION_TAU = 0.26

def set_fish_position(pos):
    data.mocap_pos[FISH_MOCAP_ID] = pos
    data.mocap_quat[FISH_MOCAP_ID] = np.array([1.0, 0.0, 0.0, 0.0])

def smoothstep(s):
    s = np.clip(s, 0.0, 1.0)
    return 3 * s**2 - 2 * s**3

def get_state(t):
    if t < BITE_TIME:
        return STATE_MONITOR
    elif t < BITE_TIME + HOOK_DURATION:
        return STATE_HOOK_SET
    else:
        return STATE_REGULATE

def fish_motion_and_pull(t, state):
    """
    Fish position is used for 3D visualization.
    Fish pull is the simulated load-cell disturbance.

    Since the current robot is only 1-DOF pitch,

```

the fish's 3D motion is treated as an external disturbance,
not as a position-tracking target.

"""

if state == STATE_MONITOR:

 fish_pos = FISH_BASE.copy()

 fish_pull = 0.0

elif state == STATE_HOOK_SET:

 s = smoothstep((t - BITE_TIME) / HOOK_DURATION)

 fish_pos = FISH_BASE + np.array([

 0.12 * s,

 0.06 * np.sin(np.pi * s),

 -0.10 * s

])

 fish_pull = 0.75 + 0.55 * np.sin(np.pi * s)

else:

 tau = t - BITE_TIME - HOOK_DURATION

3D fish-like motion:

x: forward/backward

y: left/right

z: up/down underwater

 fish_x = 1.90 + 0.18 * np.sin(0.75 * tau)

 fish_y = 0.45 + 0.18 * np.sin(0.55 * tau + 0.7)

 fish_z = -0.55 + 0.10 * np.sin(0.85 * tau + 1.4)

 fish_pos = np.array([fish_x, fish_y, fish_z])

Simulated load-cell fish pulling force.

This is not directly computed from fish position because

hardware will use an actual load cell reading.

 fish_pull = (

 1.00

 + 0.46 * np.sin(0.95 * tau)

 + 0.22 * np.sin(2.10 * tau + 0.7)

 + 0.10 * np.sin(0.38 * tau + 1.2)

)

return fish_pos, max(fish_pull, 0.0)

def add_visual_line(renderer, tip, fish, radius=0.005):

 scene = renderer.scene

if scene.ngeom >= scene.maxgeom:

return

 geom = scene.geoms[scene.ngeom]

 mujoco.mjv_initGeom(

 geom,

 mujoco.mjtGeom.mjGEOM_CAPSULE,

 np.zeros(3),

```

        np.zeros(3),
        np.eye(3).reshape(-1),
        np.array([0.0, 0.9, 0.9, 1.0])
    )

    mujoco.mjv_connector(
        geom,
        mujoco.mjtGeom.mjGEOM_CAPSULE,
        radius,
        np.asarray(tip, dtype=np.float64),
        np.asarray(fish, dtype=np.float64)
    )

    scene.ngeom += 1

def simulated_load_cell_update(t, state, fish_pull, q_actual, T_true_prev, T
    """
    Simulated load-cell signal.
    In hardware, this block will be replaced by real load-cell reading.
    """
    if state == STATE_MONITOR:
        T_target = 0.0
    else:
        # q smaller = rod raised = more line tension
        # q larger = rod lowered = less line tension
        rod_effect = ROD_TENSION_GAIN * (Q_NEUTRAL - q_actual)

        noise = 0.010 * np.sin(8.0 * t) + 0.005 * np.sin(17.0 * t + 0.4)
        T_target = max(fish_pull + rod_effect + noise, 0.0)

    beta = DT / (TENSION_TAU + DT)
    T_true = (1.0 - beta) * T_true_prev + beta * T_target

    T_filtered = (1.0 - FILTER_ALPHA) * T_filtered_prev + FILTER_ALPHA * T_t

    return T_true, T_filtered

```

In []: **class** AdmittanceController:

```

    """
    Outer loop:
        Virtual mass-spring-damper driven by force error.

    Inner loop:
        PD position tracking
        Gravity Compensation
    """

    def __init__(self, dt):

        self.dt = dt

        self.theta_cmd = Q_INIT

```

```

self.theta_dot_cmd = 0.0

# Virtual admittance parameters
self.M_v = 0.6
self.B_v = 5.5
self.K_v = 16.0

# Force scaling
self.K_f = 5.0

# Inner position tracking controller
self.Kp_track = 140.0
self.Kd_track = 20.0
self.Ki_track = 0.0

self.pos_int = 0.0

# Gravity compensation
self.K_g = 9.95

self.J_filtered = 0.0
self.J_alpha = 0.05

def get_jacobian(self, theta, tip_pos, fish_pos):
    """
    Jacobian relating rod pitch motion to line length.
    """

    L = 1.0

    line_vec = fish_pos - tip_pos
    distance = np.linalg.norm(line_vec)

    if distance < 1e-6:
        return 0.0

    u = line_vec / distance

    dx_dtheta = L * np.cos(theta)
    dz_dtheta = -L * np.sin(theta)

    return dx_dtheta * u[0] + dz_dtheta * u[2]

def update(
    self,
    t,
    state,
    f_measured,
    theta,
    theta_dot,
    tip_pos,

```

```

        fish_pos,
        control_enabled,
    ):

        if state == STATE_MONITOR or not control_enabled:
            f_desired = 0.0
        else:
            f_desired = T_DESIRED

        force_error = f_desired - f_measured

        J = self.get_jacobian(theta, tip_pos, fish_pos)

        self.J_filtered += self.J_alpha * (J - self.J_filtered)

        tau_force = -(self.J_filtered * force_error) * self.K_f

        tau_spring = self.K_v * (Q_NEUTRAL - self.theta_cmd)

        theta_ddot_cmd = (
            tau_force
            + tau_spring
            - self.B_v * self.theta_dot_cmd
        ) / self.M_v

        # Integrate

        self.theta_dot_cmd += theta_ddot_cmd * self.dt
        self.theta_cmd += self.theta_dot_cmd * self.dt

        # Keep virtual rod inside physical limits

        self.theta_cmd = np.clip(
            self.theta_cmd,
            Q_MIN,
            Q_MAX,
        )

        e_pos = self.theta_cmd - theta
        e_vel = self.theta_dot_cmd - theta_dot

        self.pos_int += e_pos * self.dt
        self.pos_int = np.clip(self.pos_int, -0.15, 0.15)

        torque_cmd = (
            self.Kp_track * e_pos

```

```

        + self.Kd_track * e_vel
        + self.Ki_track * self.pos_int
    )

    torque_cmd += -self.K_g * np.sin(theta)

    torque_cmd = np.clip(torque_cmd, -20.0, 20.0)

    return torque_cmd, self.theta_dot_cmd

def run_simulation(control_enabled=True, make_video=True):
    mujoco.mj_resetData(model, data)
    controller = AdmittanceController(dt=DT)

    data.qpos[PITCH_QPOS_ID] = Q_INIT
    data.qvel[PITCH_QVEL_ID] = 0.0
    set_fish_position(FISH_BASE)
    mujoco.mj_forward(model, data)

    # Settling loop
    for _ in range(700):
        q_actual = data.qpos[PITCH_QPOS_ID]
        q_dot_actual = data.qvel[PITCH_QVEL_ID]
        settle_torque = 50.0 * (Q_INIT - q_actual) - 5.0 * q_dot_actual - cc
        data.ctrl[0] = settle_torque
        set_fish_position(FISH_BASE)
        mujoco.mj_step(model, data)

    data.time = 0.0
    out = {"t": [], "q": [], "q_dot_cmd": [], "torque_cmd": [], "T_true": [],
          "T_filtered": [], "fish_pull": [], "state": [], "fish_pos": [],
          "rod_tip": [], "line_length": [], "frames": [], "theta_virtual":

    T_true = 0.0
    T_filtered = 0.0
    t_frame = 0.0

    while data.time < T_FINAL:
        t = data.time
        state = get_state(t)

        mujoco.mj_forward(model, data)
        q_actual = data.qpos[PITCH_QPOS_ID]
        q_dot_actual = data.qvel[PITCH_QVEL_ID]

        fish_pos, fish_pull = fish_motion_and_pull(t, state)
        set_fish_position(fish_pos)
        mujoco.mj_forward(model, data)

        tip = data.site_xpos[ROD_TIP_ID].copy()

        T_true, T_filtered = simulated_load_cell_update(
            t=t, state=state, fish_pull=fish_pull,
            q_actual=q_actual, T_true_prev=T_true, T_filtered_prev=T_filtered

```

```

)

J_actual = controller.get_jacobian(q_actual, tip, fish_pos)

if state != STATE_MONITOR:
    data.qfrc_applied[PITCH_QVEL_ID] = J_actual * T_true
else:
    data.qfrc_applied[PITCH_QVEL_ID] = 0.0

if state == STATE_HOOK_SET:
    s = smoothstep((t - BITE_TIME) / HOOK_DURATION)
    target = Q_INIT + (Q_HOOK - Q_INIT) * s

    torque_cmd = 150.0 * (target - q_actual) - 15.0 * q_dot_actual -
    q_dot_cmd = 0.0

    controller.theta_cmd = q_actual
    controller.theta_dot_cmd = q_dot_actual
    controller.pos_int = 0.0

elif state == STATE_REGULATE:
    if control_enabled:
        # Admittance Controller
        torque_cmd, q_dot_cmd = controller.update(
            t=t, state=state, f_measured=T_filtered,
            theta=q_actual, theta_dot=q_dot_actual,
            tip_pos=tip, fish_pos=fish_pos, control_enabled=True
        )
    else:
        # BASELINE: Just rigidly hold the neutral angle
        target = Q_NEUTRAL
        torque_cmd = 100.0 * (target - q_actual) - 10.0 * q_dot_actu
        q_dot_cmd = 0.0

else:
    target = Q_INIT
    torque_cmd = 100.0 * (target - q_actual) - 10.0 * q_dot_actual -
    q_dot_cmd = 0.0

data.ctrl[0] = torque_cmd
mujoco.mj_step(model, data)

line_length = np.linalg.norm(fish_pos - tip)

out["t"].append(t)
out["q"].append(np.rad2deg(q_actual))
out["q_dot_cmd"].append(np.rad2deg(q_dot_cmd))
out["torque_cmd"].append(torque_cmd)
out["T_true"].append(T_true)
out["T_filtered"].append(T_filtered)
out["fish_pull"].append(fish_pull)
out["state"].append(state)
out["fish_pos"].append(fish_pos.copy())
out["rod_tip"].append(tip.copy())

```

```

        out["line_length"].append(line_length)

        if control_enabled:
            out["theta_virtual"].append(np.rad2deg(controller.theta_cmd))
        else:
            out["theta_virtual"].append(np.nan) # Append NaN if control is r

        if make_video and data.time >= t_frame:
            renderer.update_scene(data, camera=cam)
            add_visual_line(renderer, tip, fish_pos, radius=0.005)
            out["frames"].append(renderer.render().copy())
            t_frame += DT_FRAME

    for k in out:
        if k != "frames": out[k] = np.array(out[k])

    return out
print("Control and simulation setup completed")

```

Control and simulation setup completed

```

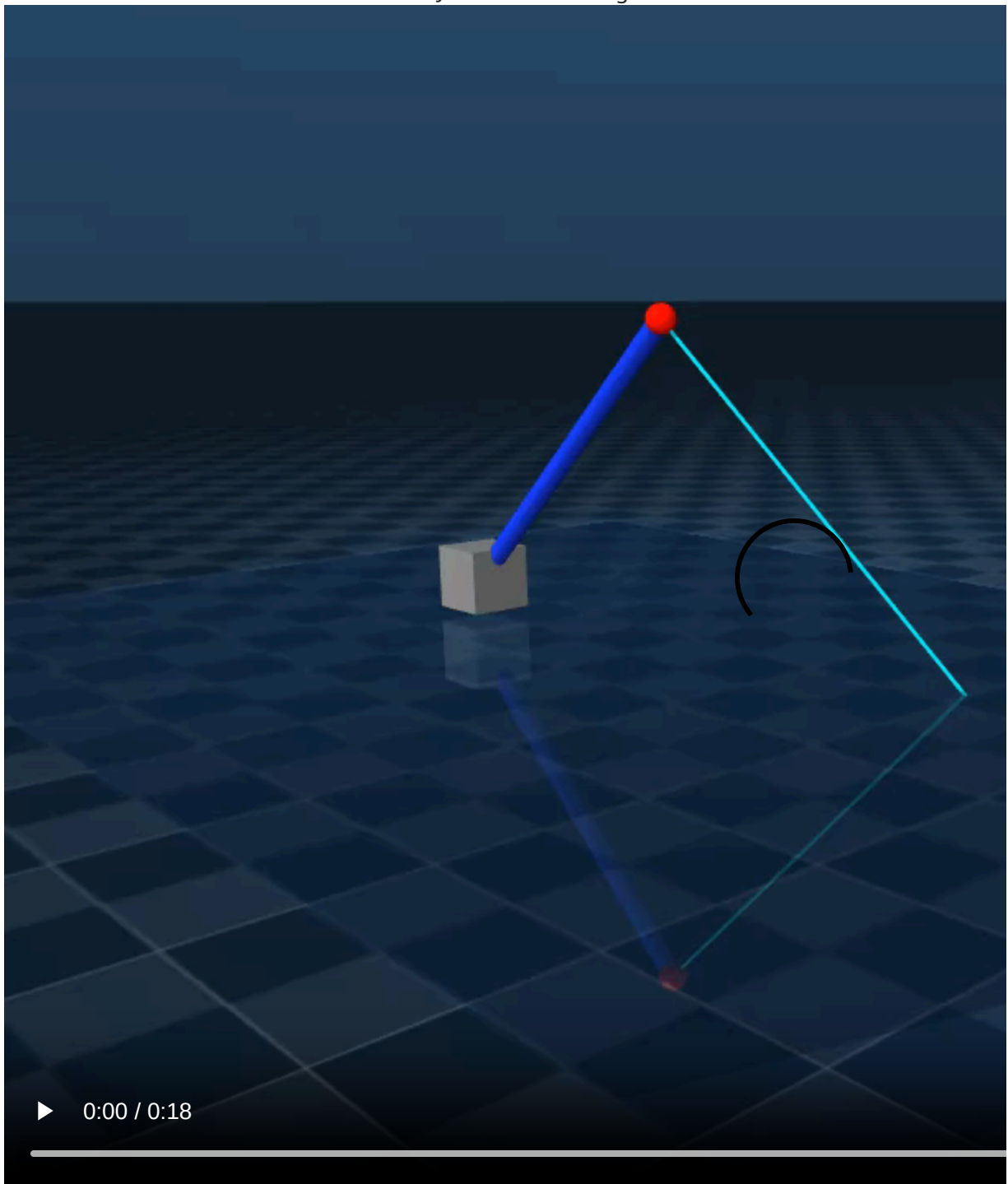
In [ ]: res_base = run_simulation(control_enabled=False, make_video=False)
        res_ctrl = run_simulation(control_enabled=True, make_video=True)

        print(f"Baseline samples: {len(res_base['t'])}")
        print(f"Controlled samples: {len(res_ctrl['t'])}")
        print(f"Controlled video frames: {len(res_ctrl['frames'])}")

        media.show_video(
            res_ctrl["frames"],
            fps=int(1 / DT_FRAME),
            title="MuJoCo 1-DOF Fishing Rod: 3D Fish Disturbance + Load-Cell Feedback"
        )

```

Baseline samples: 1800
 Controlled samples: 1800
 Controlled video frames: 451



```
In [ ]: def compute_metrics(res_base, res_ctrl):
    reg_start = BITE_TIME + HOOK_DURATION + 1.0

    mask_b = res_base["t"] > reg_start
    mask_c = res_ctrl["t"] > reg_start

    err_b = res_base["T_filtered"][mask_b] - T_DESIRED
    err_c = res_ctrl["T_filtered"][mask_c] - T_DESIRED

    rms_b = np.sqrt(np.mean(err_b**2))
    rms_c = np.sqrt(np.mean(err_c**2))
```

```

    improvement = (1.0 - rms_c / rms_b) * 100.0

    return rms_b, rms_c, improvement, reg_start

rms_b, rms_c, improvement, reg_start = compute_metrics(res_base, res_ctrl)

print(f"Baseline RMS tension error:    {rms_b:.3f}")
print(f"Controlled RMS tension error: {rms_c:.3f}")
print(f"Error reduction:                {improvement:.1f}%")

# Increased figure height to accommodate 5 subplots
plt.figure(figsize=(12, 15))

# Tension comparison
plt.subplot(5, 1, 1)
plt.plot(res_base["t"], res_base["T_filtered"],
         label="Baseline: fixed rod angle", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"], res_ctrl["T_filtered"],
         label="Controlled: admittance feedback", linewidth=2.6)
plt.axhline(T_DESIRED, linestyle="--", label="Desired tension")
plt.axhline(T_LOW, linestyle=":", label="Low bound")
plt.axhline(T_HIGH, linestyle=":", label="High bound")
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Tension")
plt.title("Simulated Load-Cell Tension Feedback")
plt.legend(loc="upper right")
plt.grid(True)

# Rod angle (showing just actual rod physics for baseline)
plt.subplot(5, 1, 2)
plt.plot(res_base["t"], res_base["q"],
         label="Baseline physical rod angle", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"], res_ctrl["q"],
         label="Controlled physical rod angle", linewidth=2.6)
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Angle from vertical [deg]")
plt.title("Physical Rod Pitch Motion")
plt.legend(loc="upper right")
plt.grid(True)

# Virtual rod angle vs Physical rod angle
plt.subplot(5, 1, 3)
plt.plot(res_ctrl["t"], res_ctrl["theta_virtual"],
         label="Virtual rod angle (cmd)", linewidth=2.6, color='purple')
plt.plot(res_ctrl["t"], res_ctrl["q"],
         label="Physical rod angle (actual)", linewidth=2.2, alpha=0.75, linestyle='solid')
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Angle from vertical [deg]")
plt.title("Admittance Controller Virtual vs. Physical Rod Angle")

```

```

plt.legend(loc="upper right")
plt.grid(True)

# Motor Torque Command
plt.subplot(5, 1, 4)
plt.plot(res_ctrl["t"], res_ctrl["torque_cmd"],
         label="Commanded Motor Torque", color="red", linewidth=2.0)
plt.axvline(BITE_TIME, linestyle=":", color="gray", alpha=0.8)
plt.axvline(BITE_TIME + HOOK_DURATION, linestyle=":", color="gray", alpha=0.8)
plt.axvline(reg_start, linestyle="--", color="gray", alpha=0.7)
plt.ylabel("Torque [Nm]")
plt.title("Admittance Controller Output (Motor Torque)")
plt.legend(loc="upper right")
plt.grid(True)

# Regulation-phase tension error only
plt.subplot(5, 1, 5)
mask_b = res_base["t"] > reg_start
mask_c = res_ctrl["t"] > reg_start

plt.plot(res_base["t"][mask_b],
         res_base["T_filtered"][mask_b] - T_DESIRED,
         label="Baseline tension error", linewidth=2.2, alpha=0.75)
plt.plot(res_ctrl["t"][mask_c],
         res_ctrl["T_filtered"][mask_c] - T_DESIRED,
         label="Controlled tension error", linewidth=2.6)
plt.axhline(0, linestyle="--", color="gray")
plt.ylabel("Tension Error")
plt.xlabel("Time [s]")
plt.title(f"Regulation Phase RMS Error Reduction = {improvement:.1f}%")
plt.legend(loc="upper right")
plt.grid(True)

plt.tight_layout()
plt.show()

```

```

Baseline RMS tension error: 0.338
Controlled RMS tension error: 0.195
Error reduction: 42.4%

```

